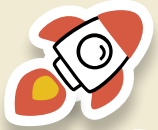
Bill Splitter
for Roommates!

Zoe Moccia, Alexander Sirotkin, Lucy Gavrich,
Naysa Arora, Niharika Elangovan,
Zainub Waqas (Project Manager)

Table of Contents



01 Data Diagram

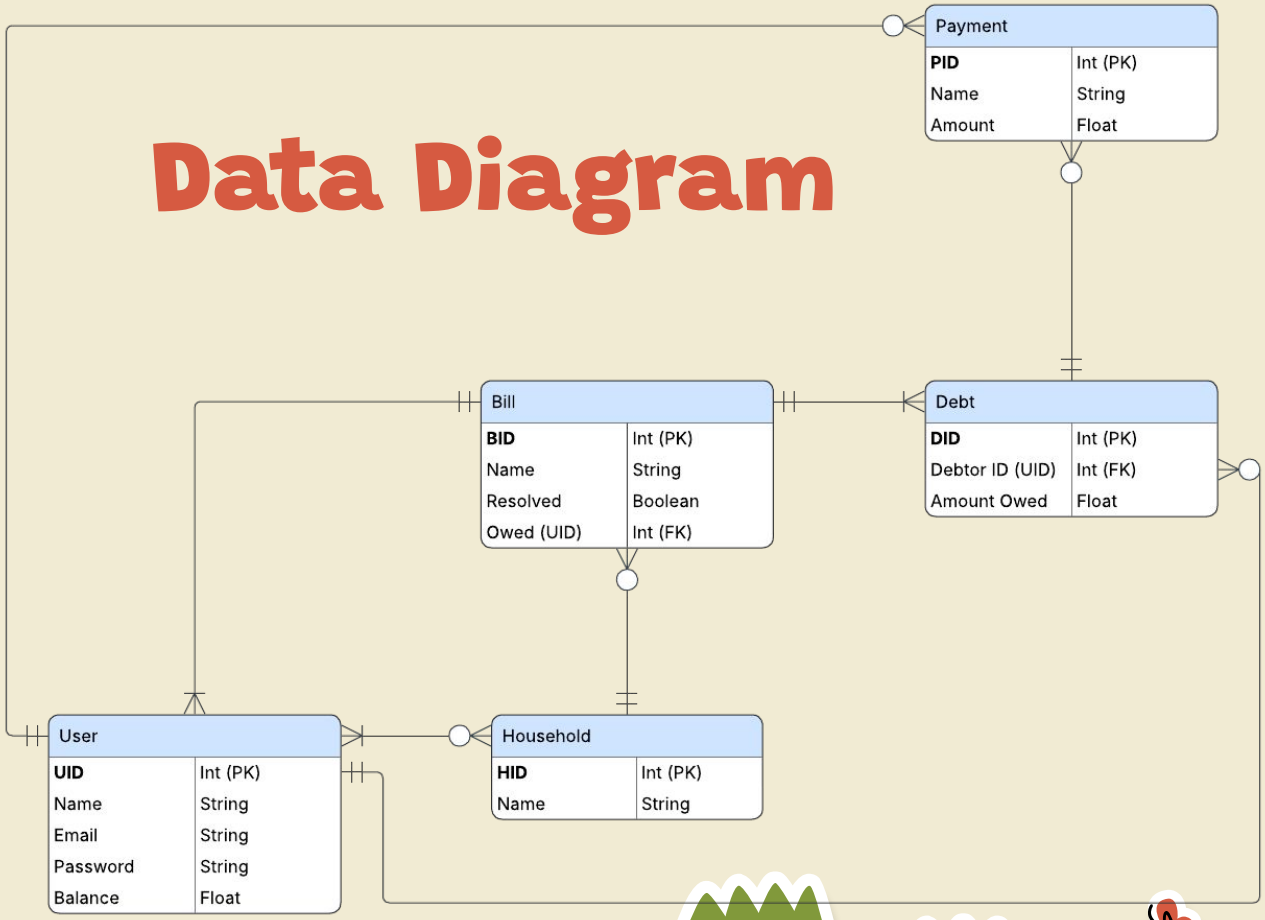
02 Frameworks

03 Endpoints

04 Function signatures

05 Testing

Data Diagram



Frameworks

- **Django** to serve API endpoints and manage DB access
- **Supabase** as the DBMS
- **React** for a dynamic front-end
- **Shadcn** components w/ **Tailwind** for design

End Point Routes

localhost/api/auth/login/ sends login data to backend and gets confirmation

- `login(email: string, password: string)`

localhost/api/households/ retrieve all households

- `getHouseholds()`

localhost/api/bills/ retrieve all active bills and debts that you are involved in

- `getBills()`

End Point Routes

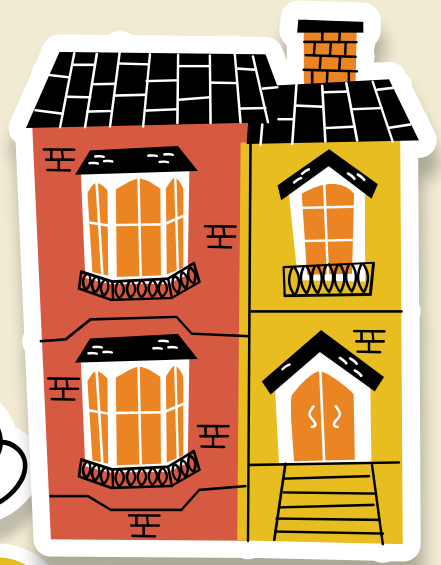
localhost/api/bills-by-person/ retrieve active bills and debts sorted by which household member other than the logged in user they are owed to/by

- `getBillsByPerson()`

localhost/api/bills-resolved/ retrieve all resolved bills and debts that you were involved in

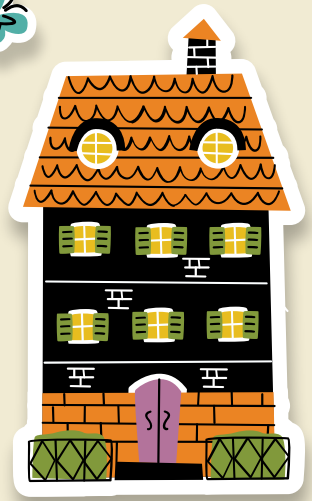
- `getResolved()`

Function signatures



```
login(email: string, password: string):
  Promise<{message: string, token: string}>

getHouseholds(): Promise<Household[]>

getBills(): Promise<{bills: Bill[], debts:
  Debt[]}>

getBillsByPerson(): Promise<[user: User]:
  {bills: Bill[], debts: Debt[]}[]>

getResolved(): Promise<{bills: Bill[],
  debts: Debt[]}>
```

Testing



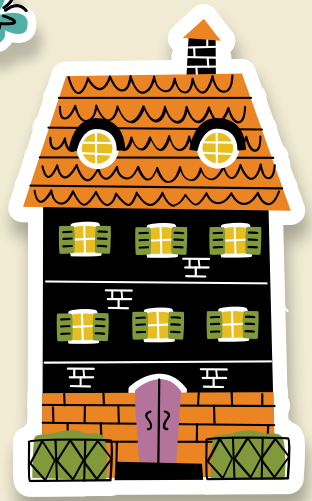
login

Populate user table with the following data:

- **user**: house@gmail.com **password**: 1234
- **user**: hold@gmail.com **password**: 5678

```
login("house@gmail.com", "1234")

login("house@gmail.com", "5678")

login("house@gmail.com", "7777")

login("person@gmail.com", "1234")

login("person@gmail.com", "7777")
```

Testing



getHouseholds

Retrieve all households a user belongs to.
Return empty array if none.

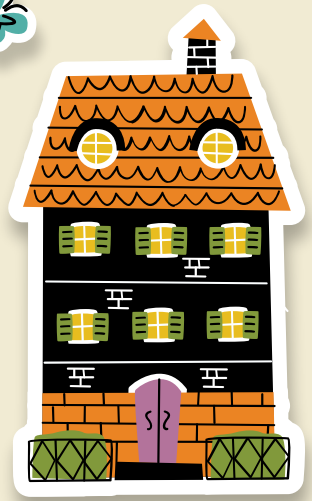
Setup:

Households table:

- H1: “131 Apartment” – members: Alice, Bob
- H2: “Maple House” – members: Alice, Charlie

User table populated with data for Alice, Bob, Charlie, Dave.

Testing



getHouseholds

Logged in user: Alice

`getHouseholds() → [H1, H2]`

Logged in user: Bob

`getHouseholds() → [H1]`

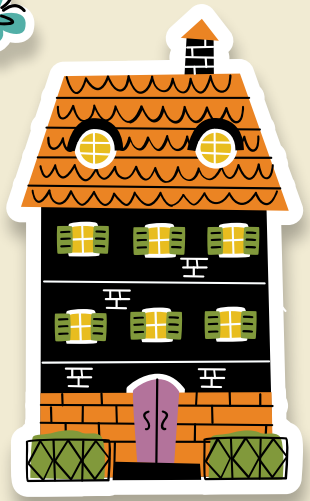
Logged in user: Charlie

`getHouseholds() → [H2]`

Logged in user: Dave (not part of any household)

`getHouseholds() → []`

Testing



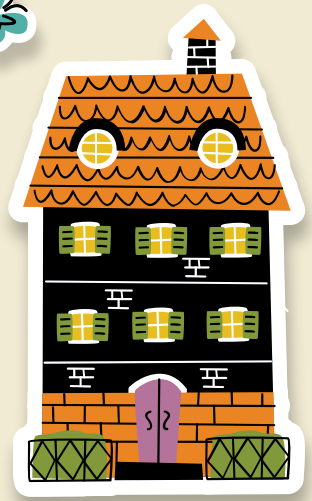
getBills

Household table populated with a new household, User table populated with data for Artichoke, Bart, Chad, and Green

Populate bills/debts tables with the following data:

- **B1** (Bill 1)
  - Creator: Artichoke
  - Debtors: Green
- **B2** (Bill 2)
  - Creator: Bart
  - Debtors: Artichoke, Green

Testing



getBills

Logged in member: Artichoke

`Household.getBills()` → [**B1**], [**B2**]

Logged in member: Bart

`Household.getBills()` → [**B2**], []

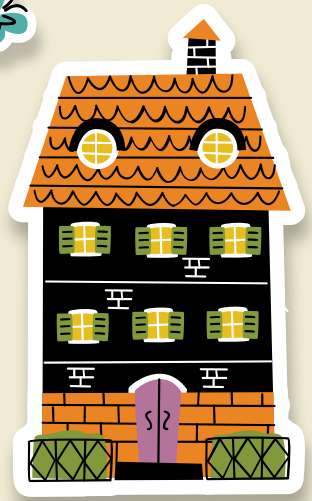
Logged in member: Chad

`Household.getBills()` → [], []

Logged in member: Green

`Household.getBills()` → [], [**B1, B2**]

Testing



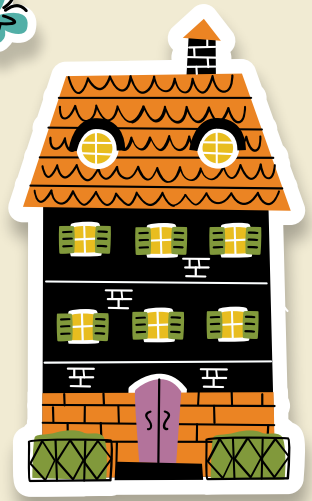
getBillsByPerson

Household table populated with same household, User table populated with data for Artichoke, Bart, Chad, and Green

We populate the Bills/debts tables with the same data:

- **B1** (Bill 1)
  - Creator: Artichoke
  - Debtors: Green
- **B2** (Bill 2)
  - Creator: Bart
  - Debtors: Artichoke, Green

Testing



getBillsByPerson

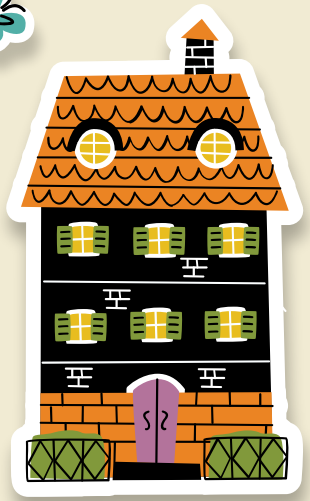
Logged in member: Artichoke

```
Household.getBillsByPerson() →
  [{Bart: ([], [B2])},
   {Chad: ([], [])},
   {Green: ([B1], [])}]
```

Logged in member: Bart

```
Household.getBillsByPerson() →
  [{Artichoke: ([B2], [])},
   {Chad: ([], [])},
   {Green: ([B2], [])}]
```

Testing



getBillsByPerson

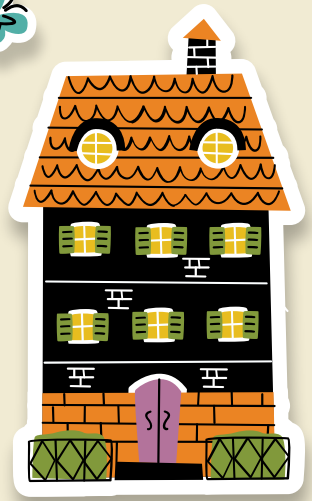
Logged in member: Chad

```
Household.getBillsByPerson() →
  [{Artichoke: ([], [])},
   {Bart: ([], [])},
   {Green: ([], [])}]
```

Logged in member: Green

```
Household.getBillsByPerson() →
  [{Artichoke: ([], [B1])},
   {Bart: ([], [B2])},
   {Chad: ([], [])}]
```

Testing



getResolved

In addition to the existing bills/debts,

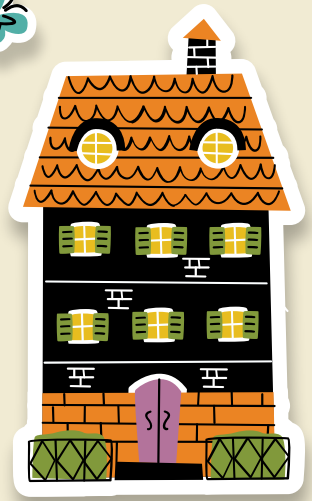
- **B1** (Bill 1)
    - Creator: Artichoke
    - Debtors: Green
- **B2** (Bill 2)
    - Creator: Bart
    - Debtors: Artichoke, Green,

add

- **B3** (Bill 3)
    - Creator: Green
    - Debtors: Bart.

Then, let **B1** and **B3** be resolved.

Testing



getResolved

Logged in member: Artichoke
`Household.getResolved()` → [**B1**], []

Logged in member: Bart
`Household.getResolved()` → [], [**B3**]

Logged in member: Chad
`Household.getResolved()` → [], []

Logged in member: Green
`Household.getResolved()` → [**B3**], [**B1**]

Thank You!